ACADEMIC CENTRAL

PRACTICE EXAMINATION

UNIVERSITY · 1ST YEAR

PROGRAMMING FUNDAMENTALS

PAPER 1 · PRACTICE A · MARKING GUIDELINE

MARKS: 100

Marking guideline for the Academic Central University · 1st year Programming Fundamentals paper 1 · practice a.

Written by Academic Central for practice, following first-year Computer Science syllabuses at South African universities (introduction to programming and algorithms). It is not a university memorandum.

How to mark: each ✓ is one mark. Accept any correct Python solution that meets the requirements, even if it differs from the model answer. Award marks for correct logic; deduct at most one mark per question for small syntax slips. Where a question bans built-in functions, award no marks for the part that uses them.

QUESTION 1

1.1 B ✓
1.2 B ✓
1.3 A ✓
1.4 C ✓
1.5 A ✓
1.6 B ✓
1.7 B ✓
1.8 B ✓
1.9 A ✓
1.10 C ✓

(10)

[10]

QUESTION 2

2.1 A compiler translates the whole program into machine code before it runs ✓; an interpreter translates and runs the program one statement at a time ✓ (2)

2.2 Sequence: statements run one after the other ✓
Selection (decision): a condition decides which statements run, e.g. if / else ✓
Repetition (iteration, loop): statements are repeated, e.g. for / while ✓ (3)

2.3 Syntax error: breaks the rules of the language, the program will not run, e.g. a missing colon ✓
Runtime error: the program crashes while running, e.g. dividing by zero ✓
Logic error: the program runs but gives the wrong result, e.g. using + instead of − ✓ (3)

2.4 A parameter is the variable named in the function definition ✓; an argument is the actual value passed when the function is called ✓ (2)

2.5 Any TWO: code can be reused instead of repeated ✓; the program is easier to read and understand ✓; easier to test and debug; work can be shared between programmers; a large problem is broken into smaller parts (2)

[12]

QUESTION 3

3.1.1 3 5 ✓ (1)

3.1.2 It swaps the values of x and y (without a temporary variable) ✓ (1)

3.2.1 i: 1, 3, 5, 7, 9 ✓
total: 1, 1 (3 is skipped), 6, 13, 13 (9 is skipped) ✓✓ (one mark off for each wrong value, minimum 0) (3)

3.2.2 13 ✓ (1)

3.3.1 13 ✓
9 ✓ (2)

3.3.2 sum_of_digits (or digit_sum) ✓ (1)

3.4 ['isak', 'BRAAI', 'ixat', 'LEKKER'] one mark for each correct item ✓✓✓✓ (4)

3.5 0 ✓ (3)
02 ✓
036 ✓

3.6 f(7) = 7 × f(5) = 7 × 5 × f(3) = 7 × 5 × 3 × f(1) = 7 × 5 × 3 × 1 ✓ (2)
Output: 105 ✓

[18]

QUESTION 4

4.1 Line 1: syntax error ✓, add a colon: def average_positive(numbers): ✓ (6)
Line 4: runtime error (range() needs a number, not a list) ✓, loop over the list itself: for n in numbers: ✓
Line 8: runtime error, division by zero when there are no positive numbers ✓, check first: if count == 0: return 0 ✓

4.2 Any normal case with the correct result, e.g. [4, −2, 8] → 6.0 ✓ (2)
An edge case with the correct result, e.g. [] → 0 or [−1, −5] → 0 ✓

4.3 Edge cases are the unusual or boundary inputs (empty list, zero, very large values) ✓ where programs most often fail, so testing them finds errors before the users do ✓ (2)

[10]

QUESTION 5

5.1 Model answer (marks shown with ✓): (5)

```
def is_leap(year):                          ✓ heading
    if year % 400 == 0:                     ✓
        return True
    if year % 100 == 0:                     ✓
        return False
    return year % 4 == 0                    ✓ correct order of tests
                                            ✓ correct True/False returned
```

5.2 Model answer: (5)

```
def count_vowels(text):                     ✓ heading
    count = 0                               ✓ counter set to 0
    for ch in text.lower():                 ✓ loop, handles upper case
        if ch in "aeiou":                   ✓ vowel test
            count += 1
    return count                            ✓ returns the count
```

5.3 Model answer: (8)

```
def trip_summary(trips):                    ✓ heading
    total = 0                               ✓
    largest = trips[0]                      ✓ start value (0 is also fine)
    small = 0
    for p in trips:                         ✓ loop over trips
        total += p                          ✓ running total
        if p > largest:                     ✓ largest updated
            largest = p
        if p < 5:                           ✓ trips under 5 counted
            small += 1
    return (total, largest, small)          ✓ tuple in the right order
```

5.4 Model answer: (12)

```
try:                                                    ✓ try / except
    f = open("marks.txt", "r")                          ✓ open the file
except FileNotFoundError:
    print("The file marks.txt was not found.")          ✓ suitable message
else:
    total = 0
    count = 0
    for line in f:                                      ✓ read every line
        number, mark = line.strip().split(",")          ✓ split on the comma
        mark = int(mark)                                ✓ convert to a number
        if mark >= 50:                                  ✓ pass test (50 or more)
            print(number, "PASS")
        else:
            print(number, "FAIL")                       ✓ both outputs
        total += mark                                   ✓ running total
        count += 1                                      ✓ count students
    f.close()
    if count > 0:
        print("Class average:", round(total / count, 1))   ✓ average to 1 decimal
                                                        ✓ program structure and indentation correct
```

[30]

QUESTION 6

6.1 After pass 1: [10, 14, 29, 13, 37] ✓ (4)
After pass 2: [10, 14, 13, 29, 37] ✓
After pass 3: [10, 13, 14, 29, 37] ✓
After pass 4: [10, 13, 14, 29, 37] (no swaps) ✓

6.2 Model answer: (8)

```
def binary_search(items, target):                   ✓ heading
    low = 0                                         ✓
    high = len(items) - 1                           ✓ bounds set correctly
    while low <= high:                              ✓ loop condition
        mid = (low + high) // 2                     ✓ middle index
        if items[mid] == target:
            return mid                              ✓ found
        elif items[mid] < target:
            low = mid + 1                           ✓ search right half
        else:
            high = mid - 1
    return -1                                       ✓ not found
```

6.3 30 ✓ 56 ✓ 61 ✓ (indices 4, 6 and 7) (3)

6.4 (a) O(n) ✓ (b) O(log n) ✓ (c) O(n^2) ✓ (3)

6.5 Binary search halves the part of the list still to be searched after every comparison ✓, so it needs at most about 20 comparisons for 1 000 000 items, while linear search may need 1 000 000 ✓ (2)

[20]

TOTAL: 100

END OF MARKING GUIDELINE